

Microduck Dance Imitation — End-to-End Evidence

- Microduck Dance Imitation: End-to-End Evidence
 - 1. Executive result
 - 2. Existing system and implementation changes
 - 3. Reference and successful training recipe
 - 4. Experiment history: failures were not hidden
 - 5. Acceptance criteria and deterministic results
 - 6. Reward and loss evidence
 - 7. Video and API verification
 - 8. Why this is genuine PPO learning
 - 9. Regression results and remaining limits
 - 10. Reproduce the test

Run date: September 7, 2026

Final run: dance-e2e-20260907-low-noise

Result: PASS for the defined, simulation-only dance-imitation test.

1. Executive result

The existing Studio API and RLX PPO implementation now have a reproducible dance-imitation test covering **real training** → **deterministic ONNX evaluation** → **rendering** → **export**, with loss telemetry, negative controls, and explicit skill acceptance.

The final policy was trained **from random initialization**, without behavior cloning, a teacher policy, joint-target feed-forward, physical assistance, or a warm-start checkpoint. PPO performed **4,001,792 environment transitions**, **1,954 rollout/update cycles**, and **31,264 optimizer minibatches**. The trained actor imitates the selected joint choreography while remaining upright.

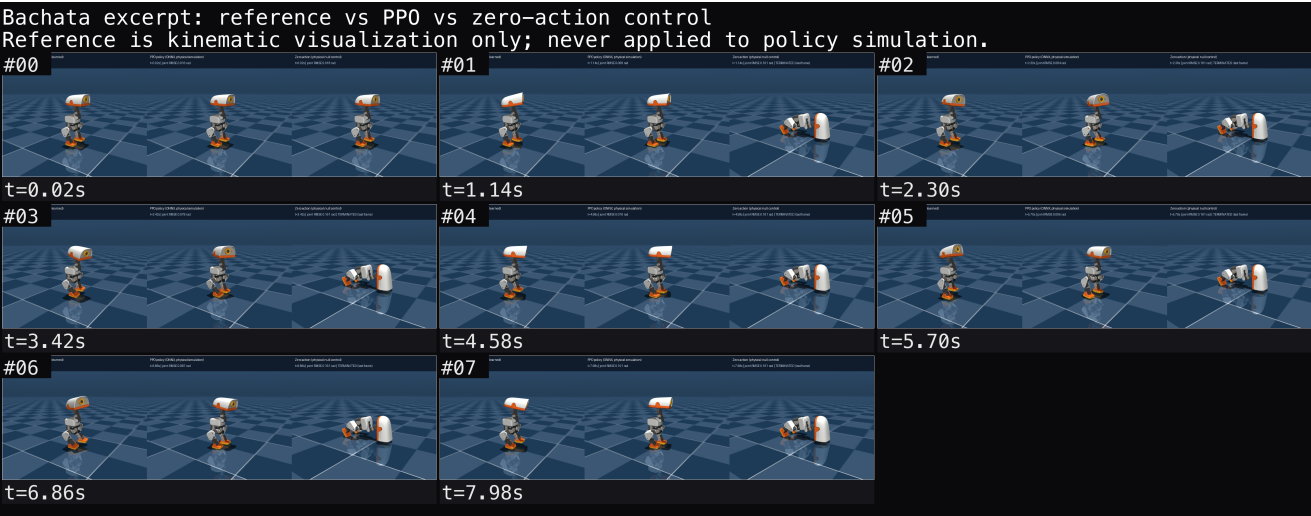
Evidence	Result
Full Studio API workflow	Train, evaluation, render, and export all succeeded
API skill evaluation	16/16 eight-second episodes passed
Separate held-out ONNX audit	5/5 eight-second episodes passed, seeds 101–105
Held-out pose error	0.08438 rad mean joint RMSE, approximately 4.8°
Held-out dynamic tracking gain	38.39% mean improvement over the best constant reference-mean pose
Held-out leg tracking gain	52.52% mean improvement over the corresponding static leg baseline
Upright coverage	100% of the primary held-out samples met the upright threshold

Evidence	Result
Extended test	3/3 sixteen-second episodes passed, seeds 201–203
ONNX/native parity	Maximum absolute action difference 3.58×10^{-7}
MP4 delivery	Decoding, frame count, duration, HTTP download, byte ranges, browser playback, and seeking verified

Scope: This is an **eight-second excerpt** of the supplied 60.923-second Bachata JSON clip. The sixteen-second test repeats that excerpt twice; it does not perform the first sixteen seconds of the original choreography. This is a successful local simulation prototype, not validation of the full source clip or authorization to deploy on hardware.

Open the evidence

- Final policy: sixteen uninterrupted seconds
- Reference versus PPO versus zero-action control: eight seconds
- Byte-verified API-delivered MP4
- Detailed audit JSON
- Complete API workflow record



The reference pane is a **kinematic illustration only**. The PPO and null panes replay states recorded from physical MuJoCo rollouts. Reference joint targets were not applied to the learned-policy simulation. The null pane explicitly holds its last frame after termination; it is not presented as a surviving eight-second rollout.

2. Existing system and implementation changes

The implementation reuses the architecture behind `rlx/examples/ppo_microduck_dance.py`, but runs the Studio entry point, `rlx/examples/ppo_microduck_studio.py`, because that is what `duck-viewer/app/api/rlx` invokes. No replacement training framework or new dependency was introduced.

User's Bachata JSON

- hashed, unscaled eight-second excerpt
- POST /api/rlx: train
- CPU MuJoCo workers + MLX PPO on Apple Metal
- checkpoint, normalizers, snapshots, JSONL telemetry, ONNX
- POST /api/rlx: eval
- complete-episode dance skill assessment
- POST /api/rlx: render → MP4 + contact sheet
- POST /api/rlx: export
- independent ONNX/control audit + browser/video verification

Problems found and addressed

Area	Original problem	Change
Reference selection	Studio factories hard-coded dance-120bpm	Optional dance_clip/--dance-clip, strict path loading, and reference SHA-256 provenance
Skill verdict	Finite dance evaluation did not establish choreography imitation	A per-episode dance assessment; pipeline and skill verdicts remain separate
UI verdict	A generic successful evaluation could be mistaken for dance skill	Dance requires explicit skill_status: passed
Pose reward	The existing broad pose kernel paid nearly its maximum for a stationary compromise	Opt-in, per-joint pose precision through dance_pose_sigma; the default remains unchanged
PPO telemetry	Studio did not persist the full optimizer metrics needed for loss plots	Actual minibatch policy/value/total losses, entropy, KL, clipping, explained variance, and JSONL journals
Reward semantics	Normalized rollout reward could be confused with raw reward	Raw episode returns/lengths are recorded separately; the UI labels normalized training reward
Reproducibility	Limited API access to the experiment's required controls	Explicit clip, horizon, rollout size, minibatches, initial standard deviation, reward normalization, and snapshot settings
Failed experiments	A failed skill evaluation could stop the test before collecting video	The HTTP driver preserves partial reports and continues render/export for failed skill evaluations, then exits nonzero

The PPO objective, Gaussian action contract, timeout bootstrapping, and observation layout were not replaced. The telemetry change preserves the existing public `update_step` return contract and is covered by parameter-parity tests with and without observation of training metrics.

Main changed files

- `rlx/examples/ppo_microduck_studio.py`: CLI plumbing, metadata, journals, checkpoints, and dance evaluation integration.
- `rlx/rlx/environments/microduck_recipes.py`: custom clips, physical metrics/state, optional precise pose reward.

- `rlx/rlx/environments/dance_evaluation.py`: strict acceptance criteria and per-episode accounting.
- `rlx/rlx/algorithms/ppo.py`: actual minibatch telemetry, without changing the optimization objective.
- `rlx/scripts/dance_e2e.py`: reference preparation, provenance, reproducible recipe generation.
- `rlx/scripts/audit_dance.py`: controls, trajectories, ONNX checks, plots, and comparative video.
- `duck-viewer/lib/rlx-job.ts`, `lib/evaluation.ts`, and `components/Studio.tsx`: API controls and honest result/reward projection.
- `duck-viewer/scripts/rlx-dance-api-e2e.mjs`: real HTTP workflow with timeout, artifact assertions, and failure reporting.
- Corresponding Python and Node regression tests, plus this report and its PDF build script.

Unrelated workspace changes, including the existing notebook/lockfile changes and observed clip deletions, were not restored or incorporated into this work.

3. Reference and successful training recipe

Reference provenance

Source: `dance-clip/bachata_microduck_v2.clip.json`.

The first eight seconds are retained without joint-amplitude scaling or re-authoring. The endpoint is interpolated on the existing 50 Hz control grid. The original looping flag is retained. The prepared reference contains 400 control-rate target poses.

The existing loader uses the fourteen joint values in the established joint order and `rootPitch`. It ignores the source's `rootYaw`, `rootRoll`, `rootPosition`, and contact annotations. Therefore, the test demonstrates **joint-pose choreography with physical balance**, not exact reproduction of an authored world trajectory, root orientation, or foot-contact sequence.

Input	SHA-256
Original Bachata JSON	306164a79a0edb494c856cda09c9f0b4a36546fcf85bb53f940ed0dc697f6a12
Eight-second reference	108bf9e56b374b36d2d86c0698bace538ac5d1db0b57d4669d94d4c739190eac
Final ONNX	32c27fe239285f1be624b01f77eaa7a411262852c89d51404ae8398ac94bdbd9

The exact input recipe is saved as `rlx/artifacts/dance-e2e-20260907/recipe-low-noise.json`. The normalized request, including all default reward weights, is retained in `api-low-noise-e2e.json`.

Parameters

Setting	Final value
Training seed	7
Transitions	4,001,792; no resume
Environment workers / backend	16 / fork

Setting	Final value
Rollout length / batch	128 control steps per worker / 2,048 transitions
Minibatches / epochs	4 / 4
Actor and critic	Separate 512–256–128 ELU MLPs
Observation / action interface	61 / 14; unchanged
Learning rate	0.0001
Discount / GAE lambda	0.99 / 0.95
PPO clip coefficient	0.2
Value coefficient / value clipping	1.0 / enabled
Maximum gradient norm	0.5
Initial action standard deviation	0.1 rad
Entropy bonus coefficient	0
Observation normalization	Enabled during training; frozen and baked into ONNX
Reward normalization	Running discounted-return normalization during training only
Episode / control frequency	8 seconds / 50 Hz
Actuator	Existing XML servo model
Domain randomization, observation noise, action delay, random yaw	Disabled for this local prototype
Snapshot interval	Approximately every 200,000 transitions, at rollout boundaries

The environment's normal seeded reset perturbations remain; disabling domain randomization does not make all reset states identical.

The pose term is:

```
pose_match = mean_j exp(-((actual_joint_j - reference_joint_j) / 0.2)^2)
weighted_pose_reward = 40 × pose_match
```

Other active weights remain: rotation matching 4, on-feet 5, no-slip penalty 0.5, no-spin penalty 0.3, gentle-head-contact penalty 1, soft-landing penalty 0.75, joint-limit penalty 1, and energy penalty 0.0005. Forward travel is disabled. The inherited `stick_it` weight is 5, but that term is inactive for a looping clip. Penalty signs are unchanged.

Runtime

Training used an **Apple M4 Pro with 64 GiB RAM**, macOS 15.7.9, CPU MuJoCo simulation, and MLX/Metal neural-network updates. No CUDA GPU was used.

The exact API training environment reported Python **3.12.10**, MLX **0.32.2**, MuJoCo **3.10.0**, NumPy **2.5.2**, ONNX **1.22.0**, ONNX Runtime **1.29.0**, Matplotlib **3.11.1**, and Gymnasium **1.3.0**. The separate audit interpreter was Python **3.12.7**, with the same listed library versions. Both runtime records are saved.

The API training job ran from **2026-09-07 23:17:29 UTC to 23:27:05 UTC**, approximately **9 minutes 37 seconds**. Instrumented collection totaled 354.85 seconds and updates 191.59 seconds; observer/checkpoint/startup overhead is excluded from those phase totals. Other diagnostic jobs overlapped, so this is an observed run time, **not an isolated performance benchmark**.

4. Experiment history: failures were not hidden

Experiment	Scope and training	Outcome
Initial smoke	4,096 transitions	Training/checkpoint/export pipeline worked; no skill claim
Broad inherited pose kernel	8 seconds, initial std 0.3, 2,000,896 transitions	Stable standing, but failed dynamic and leg tracking
Per-joint sigma 0.2	Same 8-second scope/std, 2,000,896 transitions	Better pose accuracy; still failed dynamic tracking
Lower exploration, longer budget	8 seconds, sigma 0.2, std 0.1, 4,001,792 transitions	Final successful API and held-out result
Short-sequence diagnostic	First 4 seconds, sigma 0.2, std 0.1, 1,001,472 transitions	Passed five held-out four-second trials; not the final deliverable
Precision diagnostic	8 seconds, sigma 0.1, std 0.3, 501,760 transitions	Failed dynamic tracking; not selected

The shorter diagnostic also showed that passing a short episode does not automatically establish sustained looping: its twelve-second extension failed overall. The final eight-second policy was tested separately and passed its sixteen-second extension.

A direct-reference joint-controller diagnostic fell after **0.98 seconds**. This controller was not used for training or the final policy. It illustrates why copying joint angles alone does not establish physical dance execution; PPO must also learn stabilization.

The experiment sequence does **not** prove that any single hyperparameter change was individually necessary. In particular, there was no four-million-step std-0.3 control. It establishes that the final combination works and that two million steps of the earlier recipes were insufficient. The successful policy learned substantial choreography late in training, rather than merely producing a larger early reward.

5. Acceptance criteria and deterministic results

Criteria were established before the successful run and were not relaxed to obtain a pass. A skill evaluation requires all evaluated episodes to meet all conditions:

1. Complete the configured horizon, covering at least the entire reference clip, with no fall/termination.
2. Supply finite, complete, contiguous control-rate physical state measurements.
3. Upright score at least 0.9 for at least 95% of steps.
4. Overall fourteen-joint pose RMSE at most 0.15 rad.
5. Dynamic tracking gain at least 20%, measured without fitted lag, smoothing, or time realignment.
6. At least two meaningfully moving leg joints in the reference, with positive leg tracking gain.

Moving joints have reference standard deviation at least 0.03 rad. For those joints:

$$\text{dynamic_gain} = 1 - \text{actual_tracking_RMSE} / \text{constant_reference_mean_RMSE}$$

The denominator uses each joint's mean reference pose across the evaluated episode. Thus, merely holding an approximately correct pose cannot establish a dance pass. Metrics are computed per episode; different resets cannot be pooled into an artificial complete performance.

Five held-out eight-second ONNX trials

Seed	Pose RMSE, rad	Dynamic gain	Leg gain	Upright coverage	Verdict
101	0.08123	40.77%	54.70%	100%	PASS
102	0.08275	39.76%	54.74%	100%	PASS
103	0.09827	28.05%	45.74%	100%	PASS
104	0.08148	40.50%	54.59%	100%	PASS
105	0.07816	42.86%	52.85%	100%	PASS
Mean	0.08438	38.39%	52.52%	100%	5/5

The final API evaluation also passed all sixteen eight-second lanes. Its mean pose RMSE was 0.08512 rad, dynamic gain 37.81%, and leg gain 50.08%.

Negative controls and export checks

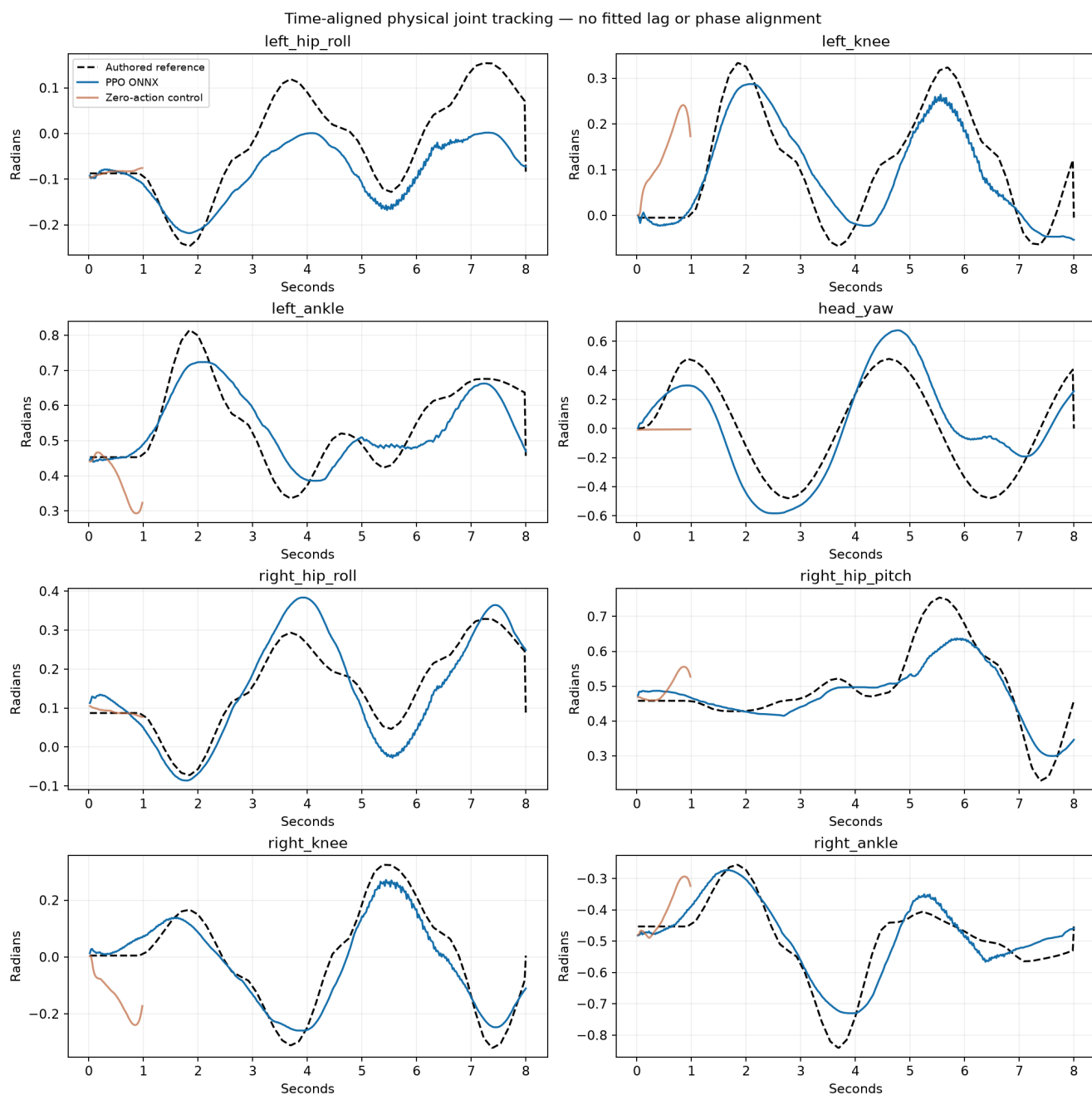
Control/check	Finding
Zero-action controller, seeds 101–105	0/5 passes; falls after 43–49 steps, or 0.86–0.98 seconds
Untrained network, seed 101	Fails after 37 steps, or 0.74 seconds
Final network with phase slots 59–60 frozen	0/5 passes; survives, but dynamic gain is negative in every trial

Control/check	Finding
Model parameter change	L2 difference 12.247 versus initial model tensors, excluding normalizer tensors
ONNX interface	Dynamic batch $\times$ 61 input; dynamic batch $\times$ 14 output; ONNX checker passes
Native/ONNX action parity	Maximum error 3.58×10^{-7} on the recorded held-out observations; tolerance 10^{-4}

Low error over an early-falling control's short prefix must not be compared as though that control completed the full choreography. This is why completion, motion, and balance criteria are mandatory alongside pose error.

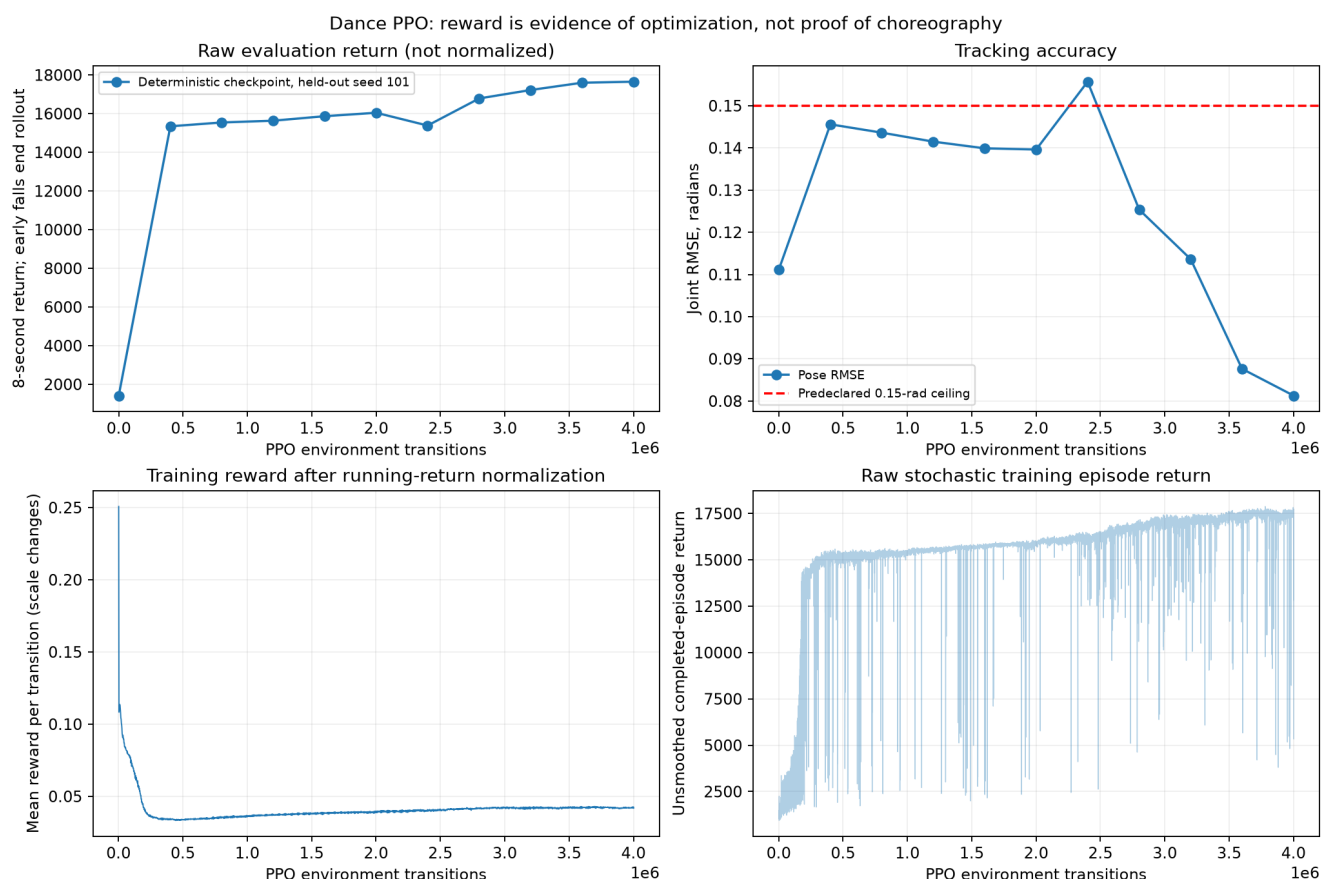
Sixteen-second extension

The final policy passed three additional uninterrupted two-cycle rollouts, seeds 201–203, with 800 control steps each. Pose RMSE was 0.08336, 0.09118, and 0.09015 rad; dynamic gains were 39.09%, 33.29%, and 34.07%. This supports two repetitions of the learned excerpt, not unlimited-duration robustness or learning unseen portions of the original source.



Visual review confirms phase-related head motion and alternating leg flexion/sway. Tracking is approximate: some hip-roll offsets, timing lag, and head-yaw amplitude overshoot remain visible. The result is not claimed to be pixel-perfect or an exact motion-capture reconstruction.

6. Reward and loss evidence

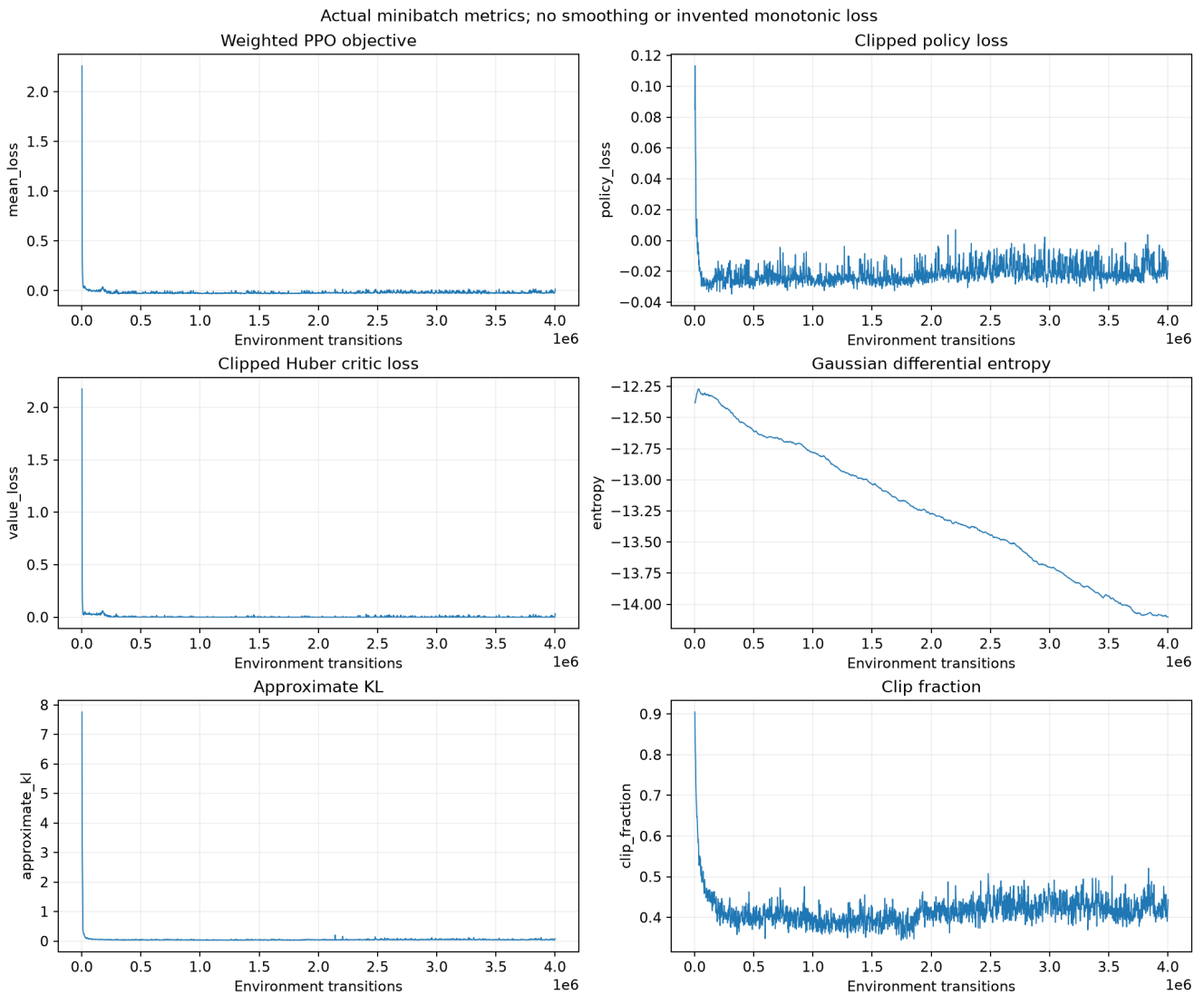


The reward figure deliberately distinguishes three quantities:

- **Raw stochastic training episode return:** the score of the exploration-noisy policy; all 12,300 completed training episodes are retained in JSONL. Short failed episodes remain visible as downward spikes.
- **Normalized rollout-buffer reward:** reward after running-return normalization. Its early decrease mainly reflects a changing normalization scale; it is not evidence that skill became worse.
- **Raw deterministic checkpoint return:** the frozen policy's physical evaluation. At seed 101, return rose from about 1,413 for the early-falling initial model to 17,654 for the final complete performance.

After initial balance learning, the policy remained near a standing compromise for a long interval. A real setback is visible around 2.4 million transitions. Dynamic gain then improved from approximately -2% at 2.0 million to 17% at 3.2 million and 36% at 3.6 million. The full run was allowed to finish; the final artifact is not a cherry-picked intermediate checkpoint.

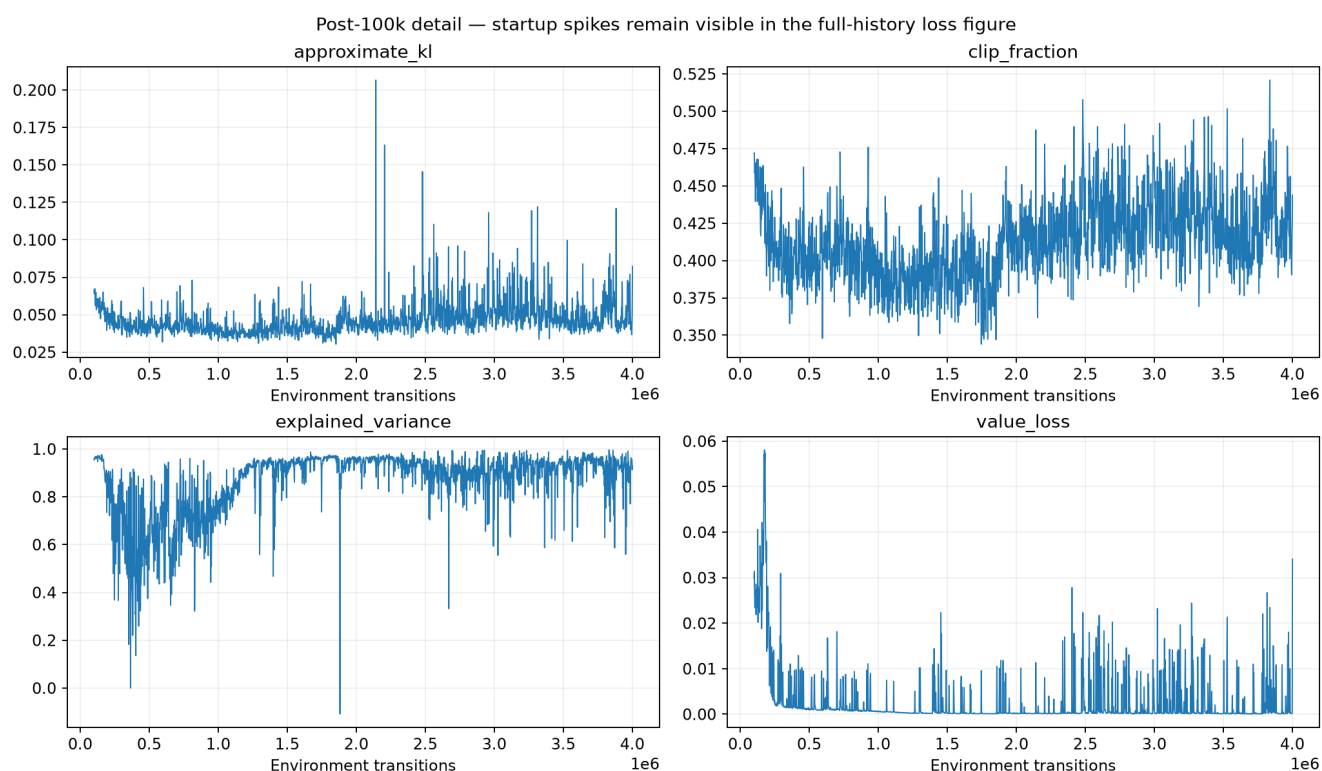
Rewards from the broad-kernel and precise-kernel experiments are **not directly comparable**, because their scoring functions differ. Physical tracking metrics and explicit skill verdicts are the meaningful cross-experiment comparison.



The loss values come from the same minibatch objective evaluations used for optimization, not a reconstructed loss or an inverted reward curve. This implementation uses the existing clipped policy objective and clipped Huber critic objective.

The first update has a large approximate KL, **7.76**, and clip fraction **0.905**. These are retained, not removed from the plots. They show an aggressive startup; this recipe is empirically successful, not claimed to be optimally tuned. No non-finite update was observed.

Across the final 100 updates, mean policy loss was -0.01673 , value loss 0.00301 , total loss -0.01372 , approximate KL 0.05317 , clip fraction 0.4306 , and explained variance 0.8951 . Loss and KL need not decrease monotonically in on-policy PPO because the sampled distribution and value targets keep changing. Negative Gaussian **differential entropy** is also valid; it does not indicate invalid probabilities.



The post-startup view is explicitly a zoom; the full-history figure retains all startup spikes. Substantial clipping remains a tuning opportunity even though the skill and finite-value checks pass.

7. Video and API verification

The full HTTP test saved each accepted normalized request and completed job state. A failed skill verdict cannot become a successful test merely because export or rendering later succeeds.

Verified eight-second videos:

Artifact	Encoding / dimensions	Frames	Duration
API policy render	H.264, 640 × 360, 25 fps	200	8.000 s
Reference/PPO/null comparison	H.264, 1440 × 412, 25 fps	200	8.000 s
Extended two-cycle policy render	H.264, 640 × 360, 25 fps	400	16.000 s

For the API-delivered video, the verification script checked:

- HTTP 200 with `video/mp4`.
- Downloaded bytes exactly equal the generated local MP4.
- HTTP 206 for `Range: bytes=0-1023`, with matching bytes.
- Chromium loaded the metadata, played beyond one second, sought to four seconds, and reported no video error.
- `ffprobe` decoded the expected frame count, duration, frame rate, and codec.

The API render records **zero resets** and the exact same ONNX SHA-256 used for evaluation. The separate sixteen-second render uses seed 201, which passed the sixteen-second physical evaluation. Its render record and contact sheet are saved alongside the video.



8. Why this is genuine PPO learning

The strongest evidence is not the reward increase alone:

1. **The initial model fails.** The final model differs in its learned tensors and completes the same physical task.
2. **The null controller fails.** The performance is not supplied by a stable spawn, passive mechanics, or gravity.
3. **Removing time information breaks imitation.** The same final actor with a frozen phase still balances, but fails dynamic tracking. It has learned a timing-dependent control policy, not just a static pose.
4. **Leg tracking improves as well as head tracking.** The leg-only acceptance prevents a head gesture from masking unmoving legs.
5. **The deterministic exported actor works.** Its results are not dependent on exploration noise, and ONNX/native parity is directly checked.
6. **The picture agrees with the numbers.** Saved physical trajectories show alternating leg motion and head motion correlated with the authored targets; videos and contact sheets were inspected.

The existing phase signal makes the target observable without changing the 61-dimensional contract. The more discriminating pose reward distinguishes precise tracking from a broad standing approximation. Reward normalization keeps critic target magnitudes manageable; the lower exploration scale supports precise actions. A sufficient training budget then lets PPO move beyond early stabilization to the timed choreography.

Those last explanations are mechanism-based interpretations supported by the observed training sequence, not a complete causal ablation of every parameter. The demonstrated claim is narrower and stronger: **this exact recorded recipe learns this defined choreography in simulation and passes the stated tests.**

9. Regression results and remaining limits

Check	Result
Local RLX suite, excluding separate mjlabs test tree	258 passed, 2 skipped
Node/API tests	37 passed
Viewer ESLint	Passed
Next.js production build and TypeScript check	Passed
Ruff correctness rules E9, F on changed Python files	Passed
Diff whitespace check	Passed
Broader microduck_local suite	384 passed, 1 skipped, 12 failed

The broader suite is **not fully green**. Its twelve failures are in unmodified code: eleven golden rollout/torque fingerprint checks and one head-versus-leg symmetry drift check. Their root cause was not isolated in this task. Golden were not regenerated, tolerances were not relaxed, and these failures are not represented as passes. Full logs are preserved in the evidence directory.

Other limits:

- One training seed and one final recipe are demonstrated; different evaluation seeds do not establish training-seed robustness.
- The final task is the eight-second excerpt, not the entire 60.923-second source or arbitrary dance clips.
- Root translation/yaw/roll and contact annotations are not imitation objectives in this existing loader.
- The local XML actuator model, disabled domain randomization, and small evaluation sample do not establish sim-to-real safety.
- Continued long-duration operation, disturbances, BAM actuator transfer, and official GPU-stack retraining remain separate work.
- The HTTP/JSON recipe is the authoritative reproduction path; the existing UI does not expose every newly supported advanced API setting as a form control.
- Training/artifact directories are gitignored and remain local. Preserve them with the report if moving the evidence to another computer.

10. Reproduce the test

Run from the workspace root on the configured Apple Silicon environment. Start the existing viewer with `cd duck-viewer && npm run dev` if it is not already running. Its default local port is 63317. Do not start a second Next.js dev server for the same directory; the attempted separate-port server was rejected by Next's directory lock, so the successful test reused the existing idle server.

```
# 1. Prepare the same unscaled eight-second reference and successful recipe.
rlx/.venv-microduck/bin/python rlx/scripts/dance_e2e.py \
  --source dance-clip/bachata_microduck_v2.clip.json \
  --output rlx/artifacts/dance-reproduction \
  --seconds 8 --train-steps 4001792 \
  --run-name dance-e2e-reproduction

# 2. Execute real train → eval → render → export through the Studio API.
node duck-viewer/scripts/rlx-dance-api-e2e.mjs \
  --execute --base-url http://127.0.0.1:63317 \
  --recipe-json rlx/artifacts/dance-reproduction/recipe.json \
  --report rlx/artifacts/dance-reproduction/api-e2e.json

# 3. Independently evaluate ONNX, controls, losses, and physical video.
rlx/.venv-microduck/bin/python rlx/scripts/audit_dance.py \
  --run rlx/runs/studio/dance/dance-e2e-reproduction \
  --recipe rlx/artifacts/dance-reproduction/recipe.json \
  --output rlx/artifacts/dance-reproduction/audit \
  --seeds 101 102 103 104 105 --history-stride 2 --render

# 4. Regression checks.
rlx/.venv-microduck/bin/python -m pytest \
  rlx/tests --ignore=rlx/tests/mjlab_microduck -q
(cd duck-viewer && npm test && npm run lint && npm run build)

# 5. Rebuild this report's PDF from its Markdown and local evidence images.
bash docs/dance-imitation-e2e/build-report.sh
```

The driver requires explicit `--execute`, has bounded operation timeouts, preserves failure evidence, and exits nonzero when full dance skill fails. The independent audit also exits nonzero if skill, finite optimizer metrics, transition coverage, model change, ONNX contract, or parity checks fail. A smoke run establishes the pipeline only, never the dance skill.

Evidence directory map

Root: `rlx/artifacts/dance-e2e-20260907/`.

- `reference-provenance.json`, `reference.clip.json`: source identity and trained target.
- `recipe-low-noise.json`, `api-low-noise-e2e.json`: exact final API recipe and workflow results.
- `runtime.json`, `training-runtime.json`: audit and training environments.
- `baseline-audit/`, `tracking-audit/`: preserved failed-policy controls, trajectories, and plots.
- `four-second/`, `precision-evaluation.json`: diagnostic experiments, not the final model.

- `final-audit/audit.json`: held-out results, controls, checkpoint learning history, optimizer accounting, and export checks.
- `final-audit/*-seed*.npz`: raw observations, actions, rewards, joint targets, actual joint positions, qpos, upright scores, and heights.
- `final-audit/reward-learning.png`, `ppo-losses.png`, `ppo-stability.png`, `joint-tracking.png`: actual-data charts.
- `final-audit/video-validation.json`, `browser-video.png`: API media delivery and playback evidence.
- `final-audit/extended-16s.json`, `render-16s.json`, `extended-render/`: independent two-cycle evaluation and video.
- `rlx-tests.log`, `node-tests.log`, `node-lint.log`, `node-build.log`, `microduck-contract-tests.log`: verification logs.

Final model and full telemetry: `rlx/runs/studio/dance/dance-e2e-20260907-low-noise/`, including `dance.safetensors`, its JSON sidecar, `dance.onnx`, `training-metrics.jsonl`, `checkpoints/`, and the original API-rendered MP4/contact sheet.